QUY HOẠCH ĐỘNG — Dynamic Programming

Chuyên đề thuật toán

Nội dung chính

1. Mở đầu
2. Tổng quát về quy hoạch động
3. Bài toán Fibonaci
4. Top-Down, Bottom-Up
5. Định nghĩa về Quy hoạch động
6. Prefix Min/Max: Cầu nối sang DP
7. Bài toán 1: Climbing Stairs
8. Bài toán 2: Frog Jump / Chi phí nhỏ nhất
9. Bài toán kinh điển: 0/1 Knapsack
10. Coin Change
11. DP trên dãy: LIS

Đối tượng & Mục tiêu chuyên đề

Đối tượng

- Học sinh lớp 9 đã có nền tảng C++ cơ bản.
- Đã học và nắm vững các thuật toán nền tảng.

Tổng kết Mục tiêu

- Hiểu được bản chất tư duy quy hoạch động (DP) ở mức ứng dụng.
- Biết nhận ra một bài toán có thể giải bằng DP.
- Thành thạo các dạng DP cơ bản trong bài thi HSG lớp 9.
- Chưa đi vào DP nâng cao (DP trên cây, đồ thị, Bitmask, Digit).

Mục tiêu: Xây dựng 5 yếu tố của DP

- Biết cách xây dựng trạng thái (`state`).
- Biết cách xây dựng công thức chuyển trạng thái (`transition`).
- Biết xác định trạng thái cơ sở (`base case`).
- Xác định đúng thứ tự tính.
- Truy xuất đáp án đúng.
- Hiểu mối liên hệ với prefix sum, prefix min/max.

Ý tưởng Triết lý trình bày

Bài toán đơn giản $\rightarrow$ đệ quy/duyệt $\rightarrow$ nhận ra tính lặp $\rightarrow$ lưu kết quả $\rightarrow$ hình thành DP $\rightarrow$ tổng quát hóa.

Một ý tưởng quen thuộc: Prefix Sum

Bài toán Mảng Cộng Dồn

Cho dãy: `a = [2, 5, 1, 7, 3]`.
Định nghĩa: $\text{prefix}[i] = a[1] + a[2] + \cdots + a[i]$.

Công thức:
$\text{prefix}[i] = \text{prefix}[i-1] + a[i]$.

Ví dụ:
`prefix = [2, 7, 8, 15, 18]`.

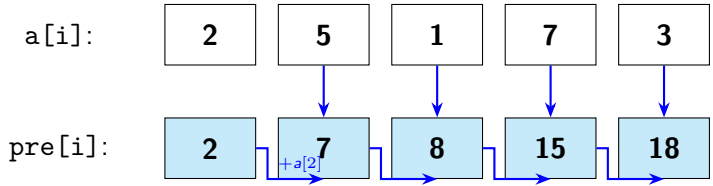
Ý tưởng Prefix Sum là tiền đề của DP

- Lưu kết quả của những trạng thái trước.
- Trạng thái mới được xây dựng từ trạng thái đã biết.
- Nhờ lưu lại, ta không phải tính lại tổng từ đầu cho mỗi truy vấn.

Mô phỏng Prefix Sum

Ghi chú Nhận xét quan trọng

Prefix Sum chưa phải một bài DP điển hình, nhưng nó là ví dụ đơn giản giúp làm quen với tư tưởng: **lưu kết quả trung gian và xây dựng kết quả mới từ kết quả đã có**.



Trạng thái mới được tạo ra từ trạng thái đã biết trước đó

Dãy Fibonacci: Bài toán quen thuộc

Định nghĩa dãy Fibonacci

$F_1 = 1, F_2 = 1$

$F_n = F_{n-1} + F_{n-2}$ (với $n > 2$)

Ví dụ

$1, 1, 2, 3, 5, 8, 13, \ldots$

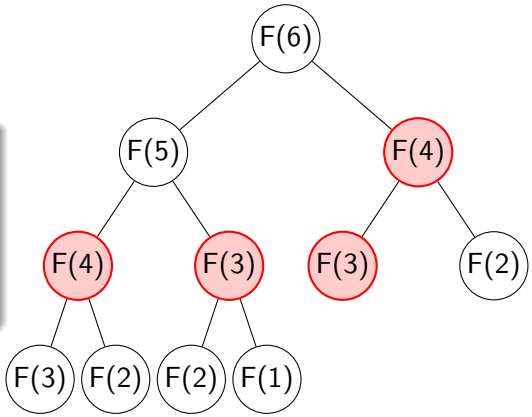
Câu hỏi đặt ra

Tính F_n với n rất lớn thì làm thế nào?

Vấn đề của Đệ quy Fibonacci trực tiếp

Vấn đề

- Công thức rất đẹp.
- Nhưng chạy rất chậm khi n lớn.
- Lý do: Cùng một giá trị bị tính đi tính lại nhiều lần.



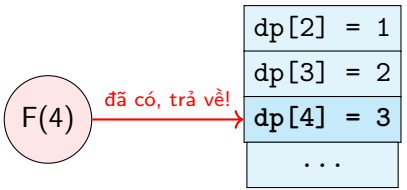
F(4), F(3)

Giải pháp: Memoization (Lưu lại kết quả)

Ý tưởng Dùng lại kết quả đã tính

- Tính $F(n)$.
- Nếu $F(n)$ chưa có $\rightarrow$ tiến hành tính toán.
- Nếu đã tính $\rightarrow$ lấy lại kết quả đã lưu trong bảng nhớ.

```
// Pseudo code Memoization
if (dp[n] da co) {
    return dp[n];
}
dp[n] = solve(n-1) + solve(n-2);
return dp[n];
```



Hai cách triển khai Quy Hoạch Động

1. Top-Down (Memoization)

Bắt đầu từ bài toán lớn, đệ quy xuống bài toán nhỏ.

$$F(n) \downarrow F(n-1), F(n-2) \downarrow \dots$$

2. Bottom-Up (Tabulation)

Bắt đầu từ trạng thái cơ sở, xây dần lên.

$$F(n) \uparrow \dots \uparrow F(3) \uparrow F(1), F(2)$$

DP Fibonacci theo kiểu Bottom-Up

```
dp[1] = 1; // Base case
dp[2] = 1;

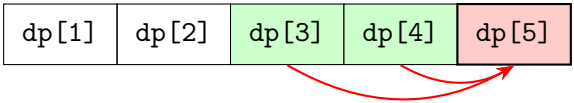
for (int i = 3; i <= n; i++) {
    dp[i] = dp[i-1] + dp[i-2];
}
```

Listing 1: Bottom-up DP

Mẹo Tư duy trạng thái

Không còn nghĩ:
"Tính F(n) bằng cách gọi F(n-1), F(n-2)."
Mà nghĩ:

"Nếu tôi đã biết dp[i-1] và dp[i-2], tôi có thể xây dp[i]."



Quy hoạch động thực sự là gì?

Tổng kết Định nghĩa trực quan

Quy hoạch động là phương pháp giải bài toán bằng cách chia thành các trạng thái nhỏ, lưu kết quả của chúng và sử dụng lại để xây dựng các trạng thái lớn hơn.

Ý tưởng Hai ý tưởng cốt lõi

1. Có các bài toán con/trạng thái có thể lặp lại.
2. Kết quả trạng thái mới có thể xây dựng từ những trạng thái đã biết.

Quy hoạch động không phải là "một công thức"

Lưu ý

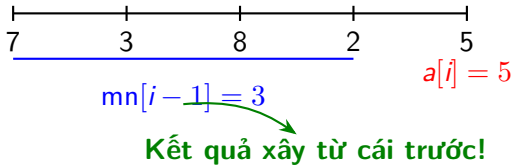
DP không phải là "Gặp DP là viết `dp[i] = ...`".

Framework 5 bước	Ý nghĩa
1. Trạng thái (State)	Định nghĩa bài toán con đang giải là gì?
2. Trạng thái trước	Trạng thái hiện tại được tạo ra từ đâu?
3. Transition	Xây dựng công thức chuyển
4. Base case & Order	Trạng thái cơ sở và thứ tự tính toán
5. Answer	Lấy đáp án từ đâu trong bảng DP?

Prefix Min / Prefix Max

Định nghĩa

Cho dãy: `a = [7, 3, 8, 2, 5]`.
Định nghĩa: $\text{mn}[i] = \min(a[1], \ldots, a[i])$.
Ta có công thức: $\text{mn}[i] = \min(\text{mn}[i-1], a[i])$.
Tương tự: $\text{mx}[i] = \max(\text{mx}[i-1], a[i])$.



Bài toán 1: Climbing Stairs (Leo cầu thang)

Đề bài

Có n bậc thang.
Mỗi lần được đi:

- 1 bậc.
- hoặc 2 bậc.

Hỏi có bao nhiêu cách đi lên bậc n?

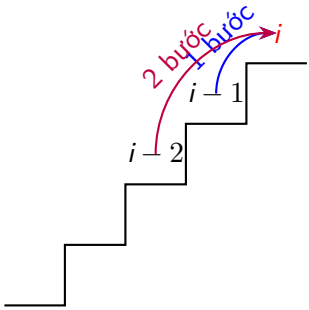
Climbing Stairs: Tìm transition bằng suy luận

Ý tưởng Xây dựng

Đặt: `dp[i]` = số cách đi đến bậc i.
Muốn đến bậc i:

- Từ $i-1$ bước 1 bậc.
- Từ $i-2$ bước 2 bậc.

Công thức:
$\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$.



Climbing Stairs: Áp dụng Framework

Tổng kết State là phần quan trọng nhất

Bước	Nội dung
State	`dp[i]` = số cách đến bậc i
Transition	`dp[i] = dp[i-1] + dp[i-2]`
Base	`dp[1]=1, dp[2]=2`
Order	i tăng dần
Answer	`dp[n]`

Bài toán 2: Frog Jump

Đề bài

Có n vị trí, mỗi vị trí có chi phí $a[i]$.
Con ếch bắt đầu ở vị trí 1.
Mỗi lần được nhảy 1 hoặc 2 bước.

Yêu cầu: Tìm tổng chi phí nhỏ nhất để đến n.

Frog Jump: Xây dựng DP Min

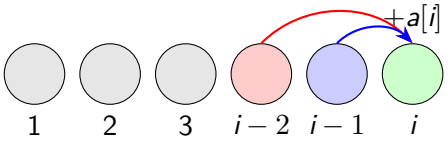
Ý tưởng Transition

Đặt: `dp[i]` = chi phí nhỏ nhất đến i.
Muốn đến i:

- Từ $i-1$.
- Hoặc từ $i-2$.

Công thức:
$\text{dp}[i] = \min(\text{dp}[i-1], \text{dp}[i-2]) + a[i]$.

Chọn đường min!



Một State có thể lưu nhiều loại thông tin

Nhìn lại Climbing Stairs và Frog Jump

`dp[i]` = số cách
`dp[i]` = chi phí nhỏ nhất
`dp[i]` = giá trị lớn nhất

Nhận xét

DP không phụ thuộc vào việc ta dùng +, `min` hay `max`.
Điều quan trọng là **state có ý nghĩa gì và transition được xây dựng thế nào**.

Bài toán 3: 0/1 Knapsack (Cái Túi)

Đề bài

Có n đồ vật.
Mỗi vật có:

- Trọng lượng $w[i]$.
- Giá trị $v[i]$.

Túi có sức chứa tối đa W.
Mỗi vật chỉ được chọn **0 hoặc 1 lần**.

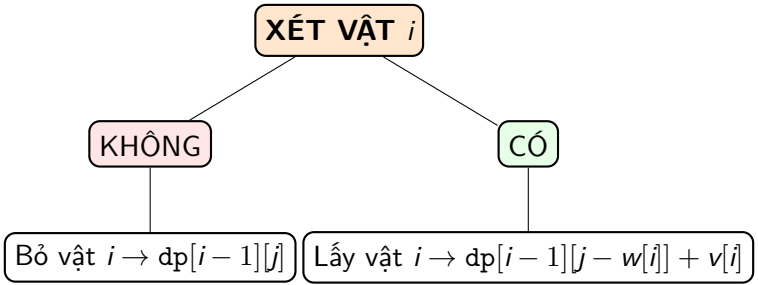
Yêu cầu: Tìm tổng giá trị lớn nhất.

Knapsack: Suy nghĩ trước khi viết công thức

Ý tưởng Chia theo lựa chọn cuối cùng

Xét vật thứ i, ta có đúng hai lựa chọn:

1. **Không lấy** vật i.
2. **Lấy** vật i (nếu còn đủ sức chứa).



State của Knapsack

Định nghĩa State

$\text{dp}[i][j]$ = giá trị lớn nhất khi xét i vật đầu tiên với sức chứa j.

Ý tưởng Transition

Nếu $w[i] \leq j$ (đủ sức chứa):
$\text{dp}[i][j] = \max(\text{dp}[i-1][j], \text{dp}[i-1][j-w[i]] + v[i])$.

Nếu không đủ chỗ ($w[i] > j$):
$\text{dp}[i][j] = \text{dp}[i-1][j]$.

Bài toán 4: Coin Change (Đổi tiền)

Đề bài

Có các loại tiền: `1, 3, 4`.
Cần tạo tổng `S`.

Yêu cầu: Tìm **số đồng tiền ít nhất**.

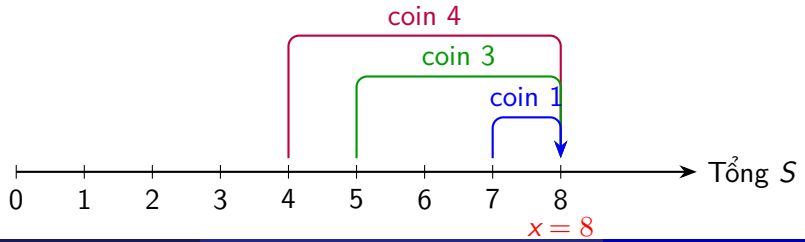
Coin Change: Xây dựng State

Ý tưởng Transition

$\text{dp}[x]$ = số đồng tiền ít nhất để tạo ra tổng x.
Muốn tạo x, đồng cuối cùng có thể là: 1, 3, hoặc 4.

Công thức:

$\text{dp}[x] = \min(\text{dp}[x-1], \text{dp}[x-3], \text{dp}[x-4]) + 1$ (với trạng thái hợp lệ).



Một công thức DP tổng quát

Công thức chung

Nếu trạng thái i có thể đến từ các trạng thái $p_1, p_2, \ldots, p_k$.
Thì công thức thường có dạng:

$$dp[i] = combine(dp[p1], dp[p2], ...)$$

- `combine` có thể là: `+`, `min`, `max`.
- Hoặc là phép `OR`/`AND` trong các bài toán tồn tại (có/không).

Bài toán 5: LIS (Longest Increasing Subsequence)

Đề bài

Cho dãy: 3 1 2 5 4 6.
Tìm **độ dài dãy con tăng dài nhất**.

Nhắc lại

"Dãy con" không nhất thiết phải liên tiếp.

Vì sao State không thể chỉ là dp[i] = đáp án toàn bộ?

Định nghĩa State đúng

dp[i] = độ dài dãy con tăng dài nhất **KẾT THÚC tại** i.

Ví dụ

Dãy: $1 \to 2 \to 5 \to 6$.
Khi xét 5, ta chỉ cần quan tâm các phần tử trước đó nhỏ hơn 5.

Kinh nghiệm thi đấu Insight quan trọng

`dp[i]` có thể chỉ mô tả một phần của bài toán, không nhất thiết là đáp án cuối cùng.

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.

3 **1** **2** **5** **4** **6**

Nối trạng thái j vào i nếu $a[j] < a[i]$

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.

3	1	2	5	4	6
dp: 1					

Nối trạng thái j vào i nếu $a[j] < a[i]$

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.

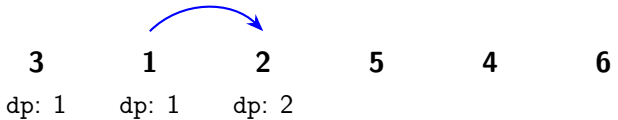


Nối trạng thái j vào i nếu $a[j] < a[i]$

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.

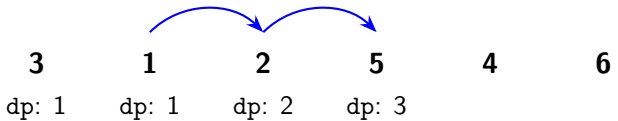


Nối trạng thái j vào i nếu $a[j] < a[i]$

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.

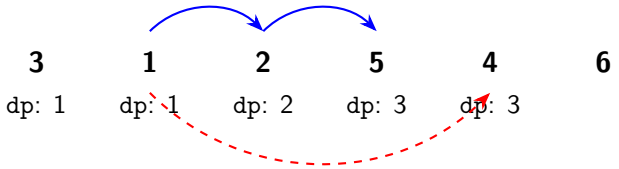


Nối trạng thái j vào i nếu $a[j] < a[i]$

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.

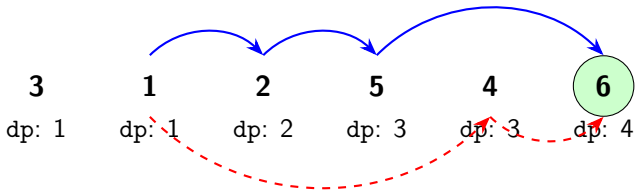


Nối trạng thái j vào i nếu $a[j] < a[i]$

Transition LIS & Mô phỏng

Ý tưởng Transition

Với mọi $j < i$ và $a[j] < a[i]$:
$\text{dp}[i] = \max(\text{dp}[i], \text{dp}[j] + 1)$.
Khởi tạo: $\text{dp}[i] = 1$.
Đáp án: $\max(\text{dp}[i])$.



Nối trạng thái j vào i nếu $a[j] < a[i]$